Modern Data Types in Python

List

A **list** is an ordered, mutable collection that allows duplicate values.

Creating a List

```
numbers = [1, 2, 3, 4, 5]
```

Accessing Elements

```
print(numbers[0])     # First element
print(numbers[-1])    # Last element
```

Slicing

```
print(numbers[1:4])   # [2, 3, 4]
print(numbers[:2])    # [1, 2]
print(numbers[3:])    # [4, 5]
```

Adding Elements

```
numbers.append(6)           # Add at end
numbers.insert(2, 10)       # Insert at index 2
numbers.extend([7, 8, 9])  # Add multiple elements
```

Removing Elements

```
numbers.remove(10)   # Remove first occurrence
numbers.pop()        # Remove last element
numbers.pop(1)       # Remove item at index 1
del numbers[0]       # Delete item at index 0
numbers.clear()      # Remove all elements
```

Updating Elements

```
numbers[0] = 100
```

Searching

```
print(3 in numbers)
print(numbers.index(3))
print(numbers.count(3))
```

Sorting

```
numbers.sort()
numbers.sort(reverse=True)
```

Reversing

```
numbers.reverse()
```

Copying

```
copy_list = numbers.copy()

# OR

copy_list = numbers[:]
```

Length

```
print(len(numbers))
```

Concatenation & Repetition

```
a = [1, 2]
b = [3, 4]

print(a + b)
print(a * 3)
```

Operation	Method/Operator
Add Item	`append(), insert()`
Add Multiple Items	`extend()`
Remove Item	`remove(), pop(), del`
Sort	`sort()`
Reverse	`reverse()`
Search	`index(), count(), in`
Copy	`copy(), [:]`
Length	`len()`
Concatenate	`+`
Repeat	`*`

Tuple

A **tuple** is an ordered, immutable collection that allows duplicate values.

Creating a Tuple

```
numbers = (1, 2, 3, 4, 5)

single = (10,)   # Single element tuple

numbers = 1, 2, 3, 4, 5
```

Accessing Elements

```
print(numbers[0])
print(numbers[-1])
```

Slicing

```
print(numbers[1:4])
```

Updating Elements

Tuples are immutable.

```
numbers[0] = 100

# TypeError
```

Adding Elements

```
numbers = numbers + (6,)
```

Removing Elements

```
numbers = (1, 2, 3, 4, 5)

numbers = numbers[:2] + numbers[3:]

print(numbers)
```

Searching

```
numbers = (1, 2, 3, 2, 5)

print(3 in numbers)
print(numbers.index(3))
print(numbers.count(2))
```

Sorting

```
numbers = (4, 2, 5, 1, 3)

sorted_numbers = tuple(sorted(numbers))
```

Reversing

```
reversed_tuple = numbers[::-1]
```

Copying

```
copy_tuple = numbers

# OR
copy_tuple = tuple(numbers)
```

Length

```
print(len(numbers))
```

Concatenation & Repetition

```
a = (1, 2)
b = (3, 4)

print(a + b)
print(a * 3)
```

Operation	Method/Operator
Access	`[]`
Slice	`[start:end]`
Search	`index(), count(), in`
Sort	`sorted()`
Reverse	`[::-1]`
Copy	`tuple()`
Length	`len()`
Concatenate	`+`
Repeat	`*`

Dictionary

A **dictionary** stores data as **key-value pairs**. Keys must be unique.

Creating a Dictionary

```
student = {
    "id": 101,
    "name": "John",
    "age": 20
}
```

Accessing Values

```
print(student["name"])

# Safer way
print(student.get("name"))
```

Adding Items

```
student["city"] = "Mumbai"
```

Updating Items

```
student["age"] = 21
```

Removing Items

```
student.pop("age")
del student["city"]
student.clear()
```

Searching

```
print("name" in student)
```

Dictionary Keys, Values and Items

```
print(student.keys())
print(student.values())
print(student.items())
```

Looping Through Dictionary

```
for key in student:
    print(key, student[key])

# OR

for key, value in student.items():
    print(key, value)
```

Copying a Dictionary

```
copy_dict = student.copy()
```

Length

```
print(len(student))
```

Merging Dictionaries

```
d1 = {"a": 1}
d2 = {"b": 2}

d1.update(d2)

print(d1)
```

Dictionary Comprehension

```
squares = {x: x*x for x in range(1, 6)}

print(squares)
```

Operation	Method
Access Value	`[], get()`
Add Item	`dict[key] = value`
Update Item	`dict[key] = value`
Remove Item	`pop(), del`
Get Keys	`keys()`
Get Values	`values()`
Get Key-Value Pairs	`items()`
Copy	`copy()`
Length	`len()`
Merge	`update()`

Set

A **set** is an unordered collection of unique elements.

Creating a Set

```
numbers = {1, 2, 3, 4, 5}

empty_set = set()
```

Adding Elements

```
numbers.add(6)

numbers.update([7, 8, 9])
```

Removing Elements

```
numbers.remove(5)     # Error if not found

numbers.discard(10)   # No error if not found

numbers.pop()         # Removes random item

numbers.clear()
```

Searching

```
print(3 in numbers)
```

Copying a Set

```
copy_set = numbers.copy()
```

Length

```
print(len(numbers))
```

Set Operations

Union

```
A = {1, 2, 3}
B = {3, 4, 5}

print(A | B)

# OR
print(A.union(B))
```

Intersection

```
print(A & B)

# OR
print(A.intersection(B))
```

Difference

```
print(A - B)

# OR
print(A.difference(B))
```

Symmetric Difference

```
print(A ^ B)

# OR

print(A.symmetric_difference(B))
```

Subset and Superset

```
A = {1, 2}
B = {1, 2, 3, 4}

print(A.issubset(B))

print(B.issuperset(A))
```

Set Comprehension

```
squares = {x*x for x in range(1, 6)}

print(squares)
```

Operation	Method/Operator
Add Item	`add()`
Add Multiple Items	`update()`
Remove Item	`remove(), discard(), pop()`
Search	`in`
Union	`union(), |`
Intersection	`intersection(), &`
Difference	`difference(), -`
Symmetric Difference	`symmetric_difference(), ^`
Subset	`issubset()`
Superset	`issuperset()`
Disjoint	`isdisjoint()`
Copy	`copy()`
Length	`len()`

String

A **string** is an **ordered**, **immutable** sequence of characters.

Creating Strings

```
name = "Python"

text = 'Programming'

message = """This is
a multi-line
string."""
```

Accessing Characters

```
name = "Python"

print(name[0])      # P
print(name[-1])     # n
```

Slicing

```
name = "Python"

print(name[1:4])    # yth
print(name[:3])     # Pyt
print(name[3:])     # hon
print(name[::-1])   # nohtyP (Reverse)
```

Updating Strings

Strings are immutable set of characters.

```
name = "Python"

# TypeError
name[0] = "J"

# To modify a string, create a new one.
name = "J" + name[1:]

print(name)
```

Concatenation

```
first = "Hello"
second = "World"

print(first + " " + second)

# Hello World
```

Repetition

```
print("Hi " * 3)

# Hi Hi Hi
```

Searching

```
text = "Python Programming"

print("Python" in text)
print(text.find("Program"))
print(text.index("Program"))
print(text.count("m"))
```

Note:

- `find()` returns `-1` if the substring is not found.
- `index()` raises a `ValueError` if the substring is not found.

Length

```
text = "Python"

print(len(text))
```

Changing Case

```
text = "Python Programming"

print(text.upper())
print(text.lower())
print(text.title())
print(text.capitalize())
print(text.swapcase())
```

Removing Spaces

```
text = "   Python   "

print(text.strip())
print(text.lstrip())
print(text.rstrip())
```

Replacing Text

```
text = "I like Java"

print(text.replace("Java", "Python"))
```

Splitting Strings

```
text = "Apple,Banana,Mango"

print(text.split(","))

# ['Apple', 'Banana', 'Mango']
```

Joining Strings

```
fruits = ["Apple", "Banana", "Mango"]

print(", ".join(fruits))

# Apple, Banana, Mango
```

Starts With / Ends With

```
text = "Python Programming"

print(text.startswith("Python"))
print(text.endswith("ing"))
```

Checking String Type

```
text = "Python123"

print(text.isalpha())      # Letters only
print(text.isdigit())      # Digits only
print(text.isalnum())      # Letters & digits
print(text.islower())
print(text.isupper())
print(text.isspace())
```

Formatting Strings

Using f-strings (Recommended)

```
name = "John"
age = 20

print(f"My name is {name} and I am {age} years old.")
```

Using format()

```
name = "John"
age = 20

print("My name is {} and I am {} years old.".format(name, age))
```

Using % Operator (Older Method)

```
name = "John"
age = 20

print("My name is %s and I am %d years old." % (name, age))
```

Escape Characters

```
print("Hello\nWorld")
print("Hello\tWorld")
print("She said \"Hello\"")
print("C:\\Users\\John")
```

String Comparison

```
print("apple" == "apple")
print("apple" != "Apple")
print("apple" < "banana")
```

Copying a String

Since strings are immutable, assigning one string to another is sufficient.

```
text = "Python"

copy_text = text

# OR

copy_text = str(text)
```

Operation	Method/Operator
Access Character	`[]`
Slice	`[start:end]`
Length	`len()`
Concatenate	`+`
Repeat	`*`
Search	`find()`, `index()`, `count()`, `in`
Replace	`replace()`
Split	`split()`
Join	`join()`
Uppercase	`upper()`
Lowercase	`lower()`
Title Case	`title()`
Capitalize	`capitalize()`
Swap Case	`swapcase()`
Remove Spaces	`strip()`, `lstrip()`, `rstrip()`
Starts With	`startswith()`
Ends With	`endswith()`
Check Alphabet	`isalpha()`
Check Digits	`isdigit()`
Check Alphanumeric	`isalnum()`
Check Lowercase	`islower()`
Check Uppercase	`isupper()`
Check Spaces	`isspace()`
Copy	`str()`

String Slicing and Indexing

```
s = "hello"
```

A string named `s` is created with the value `"hello"`.

Indexing

Each character has an index.

Character	h	e	l	l	o
Index	0	1	2	3	4
Negative Index	-5	-4	-3	-2	-1

Slicing Examples

Example 1

```
s[1:4]
```

Starts from index 1 and stops before index 4.

Output:

```
ell
```

Example 2

```
s[1:]
```

Starts from index 1 until the end.

Output:

```
ello
```

Example 3

```
s[:]
```

Prints the complete string.

Output:

```
hello
```

Example 4

```
s[1:100]
```

Python prints until the last character even if the ending index is larger than the string length.

Output:

```
ello
```

Example 5: Negative Indexing

```
s[-4]
```

Negative indexing counts from the end.

```
-5  -4  -3  -2  -1
 h   e   l   l   o
```

Output:

```
e
```

Example 6

```
s[:-3]
```

Starts from the beginning and stops before index `-3`.

Output:

```
he
```

Second String

```
s = "helloworld"
```

Index Positions

Character	h	e	l	l	o	w	o	r	l	d
Index	0	1	2	3	4	5	6	7	8	9

Reverse String

```
s[::-1]
```

Step = `-1` (Reverse)

Output:

```
dlrowolleh
```

Slice with Step

```
s[2:8:2]
```

Characters selected:

- Index 2 → `l`
- Index 4 → `o`
- Index 6 → `o`

Output:

```
loo
```

Backward Slicing

```
s[8:1:-1]
```

Starts at index 8 and moves backward until index 2.

Output:

```
lrowoll
```

Backward Slicing with Step -2

```
s[8:1:-2]
```

Output:

```
loo
```

Negative Index Slicing

```
s[-4:-2]
```

Output:

```
or
```

Arrays

Array Operations and Introduction to NumPy

1. Arrays
An array is a collection of elements of the same datatype stored in contiguous memory locations.
Example:

```
arr = [10, 20, 30, 40]
print(arr)
```

Output:
```
[10, 20, 30, 40]
```

2. Using Array Module
Python provides an array module to create arrays.from array import array

```
arr1 = array('i', [1,2,3,4,5])
print(arr1)
```

Output:
```
array('i', [1, 2, 3, 4, 5])
```

Traversing an Array

```
for i in arr1:
      print(i)
```

Output:
```
1
2
3
4
5
```

3. Array Operations

Append
Adds an element at the end of the array.

```
arr = [10,20,30]
arr.append(40)

print(arr)
```

Output:
```
[10, 20, 30, 40]
```

Insert
Inserts an element at a specific index.

```
arr.insert(1,15)
print(arr)
```

Output:
```
[10, 15, 20, 30, 40]
```

Remove
Removes a specified element.

```
arr.remove(30)
print(arr)
```

Output:
```
[10, 15, 20, 40]
```

Pop
Removes and returns the last element.

```
arr.pop()
```

Output:
```
40
```

Delete
Deletes an element using its index.

```
del arr[1]
print(arr)
```

Output:
```
[10, 20]
```

Searching in an Array

```
arr = [5,10,15,20]
print(15 in arr)
```

Output:
```
True
```

Updating an Array Element

```
arr[1] = 25
print(arr)
```

Output:
```
[5,25,15,20]
```

4. Introduction to NumPy

NumPy stands for **Numerical Python**

NumPy is a Python library used for numerical computing and working with arrays.

Features of NumPy
- Powerful N-dimensional arrays
- Mathematical operations
- Linear algebra operations
- Statistical operations
- Scientific computing

Installing NumPy
Open Command Prompt and run:
```
>  pip install numpy
```

Importing NumPy

```
import numpy as np
```

Creating a NumPy array:

```
arr = np.array([10,20,30,40])
print(arr)
```

Output:
```
[10 20 30 40]
```

5. Creating NumPy Arrays

Using np.array()

```
a = np.array([1,2,3])
print(a)
```

Output:
```
[1 2 3]
```

Using np.zeros()
Creates an array filled with zeros.

```
b = np.zeros(5)
print(b)
```

Output:
```
[0. 0. 0. 0. 0.]
```

Using np.ones()
Creates an array filled with ones.

```
c = np.ones(4)
print(c)
```

Output:
```
[1. 1. 1. 1.]
```

Using np.arange()
Creates values within a range.

```
d = np.arange(1,10)
print(d)
```

Output:
```
[1 2 3 4 5 6 7 8 9]
```

Using np.linspace()
Creates evenly spaced values.

```
e = np.linspace(1,10,5)
print(e)
```

Output:
```
[ 1.    3.25  5.5   7.75 10.  ]
```

6. Array Attributes

Example:

```
arr = np.array([[1,2,3],
                [5,4,6]])
```

ndim

Returns the number of dimensions.

```
print(arr.ndim)
```

Output:

```
2
```

shape

Returns the size of each dimension.

```
print(arr.shape)
```

Output:

```
(2,3)
```

size

Returns the total number of elements.

```
print(arr.size)
```

Output:

```
6
```

dtype

Returns the datatype of elements.

```
print(arr.dtype)
```

Output:

```
int64
```

7. Mathematical Operations on Arrays Using NumPy

Example:

```
a = np.array([1,2,3])
b = np.array([4,5,6])
```

Addition

```
print(a+b)
```

Output:

```
[5 7 9]
```

Subtraction

```
print(a-b)
```

Output:

```
[-3 -3 -3]
```

Multiplication

```
print(a*b)
```

Output:

```
[ 4 10 18]
```

Division

```
print(a/b)
```

Output:

```
[0.25 0.4  0.5 ]
```

8. Statistical Operations on Arrays Using NumPy

Example:

```
arr = np.array([10,20,30,40])
```

Sum

```
print(np.sum(arr))
```

Output:

```
100
```

Mean

```
print(np.mean(arr))
```

Output:

```
25.0
```

Maximum

```
print(np.max(arr))
```

Output:

```
40
```

Minimum

```
print(np.min(arr))
```

Output:

```
10
```

Standard Deviation

```
print(np.std(arr))
```

Output:

```
11.180339887498949
```

9. Reshaping Arrays

Example:

```
arr = np.arange(12)
print(arr)
```

Output:

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
```

Reshape the array:

```
new = arr.reshape(3,4)
print(new)
```

Output:

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

10. Array Operations

Operation	Description	Example
Traversing	Access each element one by one	`for x in arr:`
Insertion	Add a new element	`arr.append(50)`
Deletion	Remove an element	`arr.remove(20)`
Searching	Find an element	`30 in arr`
Updating	Change an existing element	`arr[2] = 100`
Sorting	Arrange elements in order	`arr.sort()`
Length	Count total elements	`len(arr)`

11. Python List vs NumPy Array

Feature	Python List	NumPy Array
Speed	Slower	Faster
Memory Usage	More	Less
Mathematical Operations	Limited	Powerful
Data Type	Mixed types allowed	Same data type
Dimensions	Mainly 1D	Multi-dimensional
Scientific Computing	Not suitable	Highly suitable
Performance	Moderate	High

12. Common NumPy Functions

Function	Purpose	Example
`np.array()`	Create a NumPy array	`np.array([1,2,3])`
`np.zeros()`	Create an array of zeros	`np.zeros(5)`
`np.ones()`	Create an array of ones	`np.ones(4)`
`np.arange()`	Create an array with a range of values	`np.arange(1,10)`
`np.linspace()`	Create evenly spaced values	`np.linspace(1,10,5)`
`np.sum()`	Find the sum of elements	`np.sum(arr)`
`np.mean()`	Find the average	`np.mean(arr)`
`np.max()`	Find the maximum value	`np.max(arr)`
`np.min()`	Find the minimum value	`np.min(arr)`
`np.reshape()`	Change the array shape	`arr.reshape(2,3)`

Array Slicing & Indexing

1. NumPy Array Slicing

```
import numpy as np

arr = np.array([1,2,3,4,5,6,7])
```

Array Indices

Index	0	1	2	3	4	5	6
Value	1	2	3	4	5	6	7

Slicing Examples

Operation	Code	Output
Elements from index 1 to 4	`arr[1:5]`	`[2 3 4 5]`
Elements from index 4 to the end	`arr[4:]`	`[5 6 7]`
Elements from the beginning to index 3	`arr[:4]`	`[1 2 3 4]`

*In Python slicing, the **starting index is included** and the **ending index is excluded**.*

2. Mathematical Functions

```
import math
```

Function	Purpose	Example	Output
`math.pow()`	Calculates power	`math.pow(2,4)`	`16.0`
`math.sqrt()`	Calculates square root	`math.sqrt(25)`	`5.0`
`math.ceil()`	Rounds upward	`math.ceil(2.8)`	`3`
`math.floor()`	Rounds downward	`math.floor(2.8)`	`2`
`math.fabs()`	Finds absolute value	`math.fabs(-5)`	`5.0`
`math.sin()`	Calculates sine (in radians)	`math.sin(45)`	`0.8509` (approx.)
`math.floor(math.sin())`	Finds floor of sine value	`math.floor(math.sin(45))`	`0`

*(`math.sin()` uses **radians**, not degrees.)*

3. Array Aliasing and Copying

Original List

```
original = [1,2,3,4]
```

Aliasing

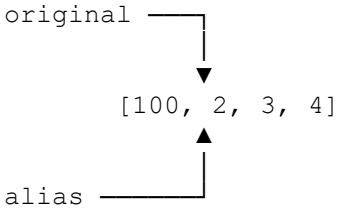
```
alias = original
alias[0] = 100

print(original)
print(alias)
```

Output

```
[100, 2, 3, 4]
[100, 2, 3, 4]
```

*Both `original` and `alias` refer to the **same list** in memory. Any change made through one variable is reflected in the other.*

Memory Representation



Copying

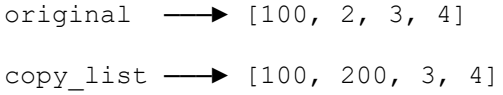
```
copy_list = original.copy()
copy_list[1] = 200

print("Original:", original)
print("Copy:", copy_list)
```

Output

```
Original: [100, 2, 3, 4]
Copy:     [100, 200, 3, 4]
```

*`copy()` creates a **new list** in memory. Changes made to the copied list do **not** affect the original list.*

Memory Representation



Difference Between Aliasing and Copying

Feature	Aliasing	Copying
Memory Location	Same object	Different objects
Changes Affect Original	Yes	No
Syntax	`alias = original`	`copy_list = original.copy()`
Use Case	Share the same data	Create an independent copy